

Prediction Task Documentation

Disposable Income Regression

For Federated Learning with Data Heterogeneity

1. Executive Summary

This document describes a regression task for predicting Disposable Income in a federated learning context. The task is designed to demonstrate the impact of data heterogeneity (Non-IID data distribution) on model performance, making it ideal for comparing federated optimization algorithms like FedAvg, FedProx, and SCAFFOLD.

Attribute	Details
Task Type	Regression
Target Variable	Disposable_Income (continuous)
Dataset Size	20,000 samples
Input Features	14 features (demographics + expenses)
Evaluation Metrics	RMSE, MAE, R ²
Non-IID Partitioning	By City_Tier (3 clients)

2. Problem Definition

2.1 Objective

Predict a user's Disposable Income based on their demographic characteristics and spending patterns across various expense categories. This task serves as a practical application of personalized financial modeling while demonstrating the challenges of federated learning under data heterogeneity.

2.2 Target Variable

Variable Name: Disposable_Income

Type: Continuous (float)

Definition: The amount of money remaining after all expenses are deducted from income.

Formula: Disposable_Income = Income - Total_Expenses

Statistic	Overall	Tier_1	Tier_3
Mean	\$10,647	\$7,156	\$13,718

Std Dev	\$11,741	\$7,934	\$14,025
Min	-\$5,401	-\$5,401	-\$3,200
Max	\$377,060	\$103,065	\$274,731

2.3 Why This Task?

1. Natural Data Heterogeneity: City_Tier creates significant variance in disposable income distributions across clients.
2. Avoids Discretization Issues: Regression eliminates the arbitrary threshold problems that caused your 3-class classification to fail.
3. Clear Metrics: RMSE and MAE provide intuitive, comparable results across IID and Non-IID experiments.
4. Real-World Application: Directly relevant to personalized financial planning and budgeting tools.

3. Feature Engineering

3.1 Input Features

The model uses 14 input features categorized into demographics and expense categories:

Demographic Features (4)

Feature	Type	Description
Age	Integer	User age (18-64)
Dependents	Integer	Number of dependents (0-4)
Occupation	Categorical	Professional, Retired, Self_Employed, Student
City_Tier	Categorical	Tier_1, Tier_2, Tier_3 (urban classification)

Expense Features (10)

Feature	Description
Rent	Monthly rent/housing payment
Loan_Repayment	Monthly loan obligations
Insurance	Insurance premiums
Groceries	Food and household supplies
Transport	Transportation costs
Eating_Out	Restaurant and dining expenses
Entertainment	Entertainment spending
Utilities	Utility bills
Healthcare	Medical expenses
Education	Education-related costs

3.2 Excluded Features

The following features are intentionally EXCLUDED from the input to prevent data leakage:

- Income - Directly determines target; would make prediction trivial
- Miscellaneous - Often a catch-all that correlates too strongly with target
- Desired_Savings_Percentage - Derived from disposable income
- Desired_Savings - Derived from disposable income
- All Potential_Savings_* columns - Post-hoc calculations

3.3 Preprocessing Steps

5. Numerical Scaling: Apply StandardScaler to all numerical features
6. Categorical Encoding: One-hot encode Occupation (4 categories) and City_Tier (3 categories)
7. Final Feature Count: 12 numerical + 7 one-hot = 19 total features

4. Data Heterogeneity Analysis

4.1 Non-IID Partitioning Strategy

The dataset is partitioned by City_Tier to create natural data heterogeneity across federated clients:

Client	City_Tier	Samples	Mean Target	Std Dev
Client 1	Tier_1	5,934	\$7,156	\$7,934
Client 2	Tier_2	10,068	\$11,486	\$12,124
Client 3	Tier_3	3,998	\$13,718	\$14,025

4.2 Source of Heterogeneity

The primary driver of heterogeneity is the Rent feature, which varies significantly by city tier:

- Tier_1 (Metropolitan): Mean rent = \$12,321 (highest cost of living)
- Tier_2 (Urban): Mean rent = \$8,343 (moderate cost of living)
- Tier_3 (Suburban/Rural): Mean rent = \$6,305 (lowest cost of living)

This creates a scenario where users in Tier_1 cities have similar incomes but lower disposable income due to higher living costs, resulting in different feature-target relationships across clients.

4.3 IID Baseline

For comparison, an IID partition randomly shuffles all 20,000 samples and distributes them evenly across 3 clients (~6,667 samples each), ensuring each client has similar data distributions.

5. Model Architecture

5.1 Neural Network Design

A simple feedforward neural network suitable for federated learning:

Layer	Type	Output Size	Activation
Input	Dense	19	-
Hidden 1	Dense	64	ReLU
Hidden 2	Dense	32	ReLU
Output	Dense	1	Linear

5.2 Training Configuration

Parameter	Value
Loss Function	Mean Squared Error (MSE)
Optimizer	SGD or Adam
Learning Rate	0.01 (local), varies by algorithm
Local Epochs	5
Batch Size	32
Global Rounds	50-100

6. Evaluation Metrics

6.1 Primary Metrics

Root Mean Squared Error (RMSE)

$$\text{RMSE} = \sqrt{\text{mean}((y_{\text{true}} - y_{\text{pred}})^2)}$$

Penalizes large errors more heavily; measured in dollars.

Mean Absolute Error (MAE)

$$\text{MAE} = \text{mean}(|y_{\text{true}} - y_{\text{pred}}|)$$

Average absolute prediction error in dollars; more robust to outliers.

R² (Coefficient of Determination)

$$R^2 = 1 - (\text{SS}_{\text{res}} / \text{SS}_{\text{tot}})$$

Proportion of variance explained by the model; 1.0 is perfect, 0.0 is baseline.

6.2 Expected Results

Based on the data characteristics, expected performance ranges:

Scenario	RMSE Range	R ² Range
IID (FedAvg)	\$3,000 - \$5,000	0.75 - 0.85
Non-IID (FedAvg)	\$5,000 - \$8,000	0.55 - 0.70
Non-IID (FedProx)	\$4,000 - \$6,000	0.65 - 0.78
Non-IID (SCAFFOLD)	\$3,500 - \$5,500	0.70 - 0.82

Note: These are estimated ranges. Actual results will depend on hyperparameter tuning and implementation details.

7. Implementation Guide

7.1 Data Loading

```
import pandas as pd
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer

# Load data
df = pd.read_csv('data_2.csv')

# Define features
numerical_features = ['Age', 'Dependents', 'Rent', 'Loan_Repayment',
                      'Insurance', 'Groceries', 'Transport', 'Eating_Out',
                      'Entertainment', 'Utilities', 'Healthcare', 'Education']
categorical_features = ['Occupation', 'City_Tier']

# Target
y = df['Disposable_Income'].values
```

7.2 Non-IID Partitioning

```
def partition_by_city_tier(df):
    partitions = {}
    for tier in ['Tier_1', 'Tier_2', 'Tier_3']:
        partitions[tier] = df[df['City_Tier'] == tier]
    return partitions
```

7.3 Flower Client Setup

```
import flwr as fl
import torch
import torch.nn as nn

class DisposableIncomeModel(nn.Module):
    def __init__(self, input_dim=19):
        super().__init__()
        self.layers = nn.Sequential(
            nn.Linear(input_dim, 64),
            nn.ReLU(),
            nn.Linear(64, 32),
            nn.ReLU(),
```

```

        nn.Linear(32, 1)
    )

    def forward(self, x):
        return self.layers(x)

```

8. Research Contributions

This prediction task enables the following research contributions:

8. Quantifying the impact of data heterogeneity on federated regression models
9. Comparing convergence rates of FedAvg, FedProx, and SCAFFOLD under Non-IID conditions
10. Demonstrating privacy-preserving financial modeling across distributed clients
11. Analyzing per-client performance to identify fairness implications

9. Appendix: Dataset Statistics

9.1 Full Feature Correlations with Target

Feature	Correlation
Income	0.681 (excluded)
Groceries	0.645
Healthcare	0.638
Transport	0.631
Utilities	0.612
Eating_Out	0.598
Rent	0.589
Insurance	0.584
Entertainment	0.576
Loan_Repayment	0.291
Dependents	0.012
Age	0.002